A Machine Learning-Based Detection of Test Automation Flakiness

Mohamed ElGazzar(✉), Eman Hossny, and Fatma A. Omara

Faculty of Computers and Information, Cairo University, Cairo, Egypt
{mohamed.elgazzar,e.hossny,f.omara}@fci-cu.edu.eg

Abstract. Test automation is essential to ensure software reliability and speed up delivery cycles. However, flaky tests pose a significant challenge by giving inconsistent results even under identical conditions. Although previous studies have focused primarily on studying flakiness within unit tests and using open-source datasets, there is a gap in addressing flakiness in integration and system testing and within commercial software environments. This research aims to fill this gap. This paper presents a study on detecting flaky tests using machine learning (ML) techniques, leveraging a dataset compiled from six commercial projects with over 2500 Java-based test cases. We evaluated several ML models, including Decision Tree, Random Forest, Support Vector Machine and Neural Networks, and demonstrated that the Random Forest and Neural Networks models achieved the highest precision, recall, F1 scores and AUC-ROC values. These results highlight the effectiveness of ML models in improving flaky test detection and underscore their importance in commercial software environments. The proposed framework offers valuable information for improving the reliability of software testing, providing practical knowledge for researchers and practitioners in the field of software engineering.

Keywords: Test Automation Stability · Regression Test Stability · Test Flakiness Detection · and Flakiness Prediction

1 Introduction

In the rapidly evolving world of software development, ensuring the reliability of automated testing is critical to maintain high-quality systems and validate software functionality, and ensure that it meets quality standards before reaching end users [1]. One of the most persistent challenges in test automation is the presence of flaky tests, tests that produce inconsistent results even under identical conditions. These flaky tests can undermine the effectiveness of test automation, slowing the development process and increasing maintenance costs [2]. This leads to wasted developer time investigating false positives and undermines trust in the automation process [3].

Traditional methods for addressing test automation flakiness typically involve rerunning failed tests. Although straightforward, these methods are resource-intensive and consume both time and computing power [4]. Despite their sim-

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
A. Abdelgawad et al. (Eds.): ISBCom 2025, LNNS 1801, pp. 91–102, 2026.
https://doi.org/10.1007/978-3-032-15784-3_8

Copyrighted material

plicity, these methods fail to address the root cause of flakiness and are inefficient, particularly in large-scale projects. Recent research has proposed various techniques for identifying flaky tests, with a predominant focus on unit tests and open source datasets, such as those of library projects [5,6]. However, these studies leave significant gaps in understanding flakiness in real-world commercial environments, particularly concerning integration and system tests. The flakiness in these types of test often arises from complex interactions within the testing environment, such as issues with concurrency, external service dependencies, and specific timing requirements. Current methods do not adequately address these challenges [7].

Our previous research identified several critical gaps in the existing literature, such as the lack of studies using industrial project data sets and the need to focus on integration and system tests rather than unit tests [8]. In light of these gaps, machine learning (ML) presents a promising alternative to traditional methods. ML can predict and identify flaky tests by analyzing historical test execution data, detecting patterns, and recognizing anomalies that signal flakiness. Unlike traditional methods, ML-based approaches do not require repeated test executions, significantly reducing resource consumption [9].

In this study, we introduce a machine learning approach for detecting flaky tests, specifically for integration and system tests in commercial software environments. Our approach leverages ML algorithms to analyze historical test execution data and learn patterns indicative of flaky behavior. By focusing on real-world datasets and commercial projects, this work aims to contribute to a more efficient and scalable solution to flaky test detection, ultimately improving the reliability of test automation frameworks.

1.1 Background

For software testing, flaky tests represent a significant problem due to their non-deterministic nature, which means that a flaky test might fail or pass during execution without any changes in the code or environment [10]. This unpredictability renders flaky tests unreliable indicators of genuine software regressions, greatly complicating the testing process. Consequently, productivity decreases, and there is a potential loss of trust in automated testing systems. Furthermore, dealing with flaky tests incurs higher computational costs, as multiple executions are necessary to determine whether a test is truly flaky or resilient enough. Durieux et al. [11] found that flaky tests impair continuous integration, causing productivity loss and inefficient tests not only in open source projects, but also in commercial projects such as Google [12], Microsoft [13], Apple [14], and Facebook [10]. A survey by Parry et al. [2] revealed that at least 56% of developers observed at least one instance of flaky behavior per month in their current workloads. Therefore, understanding and effectively detecting test automation flakiness is crucial for the success of software testing.

Detecting test automation flakiness involves various techniques and approaches. One commonly used approach [3] is the analysis of test failure patterns. By closely examining these patterns, testers can identify recurring issues or

trends that indicate flakiness, helping to understand its root causes and develop targeted solutions.

Another approach [15] is to compare the test results in different test environments. By running the same tests in multiple environments, testers can determine whether the flakiness is limited to a specific set-up or occurs consistently across all environments. This comparison helps to pinpoint environmental factors that contribute to flakiness, enabling the creation of environment-specific fixes.

Moreover, the use of statistical analysis techniques [8] can aid in detecting flaky tests in automation. Testers can apply statistical methods to analyze test results, such as calculating the mean and variance of execution times or examining the distribution of test failures. These statistical insights provide quantitative measures of flakiness, allowing testers to set thresholds or define acceptable levels of flakiness.

2 Related Work

2.1 Traditional Flaky Test Detection Approaches

Traditional flaky test detection methods often rely on relatively simple strategies to identify inconsistencies or irregularities in test results.

General Detection Strategies for flaky tests include several methods that aim to identify inconsistencies in test results. **Repeated Execution** involves running tests multiple times to observe inconsistent outcomes. This method is widely adopted in organizations such as Google and Microsoft. While effective, it is time-consuming, and studies have shown that although a significant percentage of flaky tests can be identified through this method, it incurs high execution costs. **Differential Coverage** identifies flaky tests by comparing the test coverage across different software versions. If a test fails without covering modified code, it is likely flaky. The tool *DeFlaker* implements this method and has shown high effectiveness in identifying flaky tests without the need for repeated executions. In a dataset of 28,068 test methods from 26 open-source Java projects, it identified 96% of flaky tests previously detected by rerunning tests.

Non-deterministic Specifications refers to tools like *NonDex* that identify flaky tests caused by assumptions about deterministic implementations of non-deterministic specifications. These tools randomize implementations to expose flaky behavior. Using a dataset from 21 open-source Java projects, *NonDex* identified 60 flaky tests. **Stress Testing** techniques such as *Shaker* introduce CPU and memory stress during test execution to increase the likelihood of detecting flaky tests, especially in areas like asynchronous waits and concurrency. In a dataset of 11 Android projects, *Shaker* detected flaky tests faster than standard reruns. Lastly, **Static Analysis** employs pattern matching techniques to identify flaky tests by analyzing the test code statically. It searches for known

Copyrighted material

patterns like hard-coded time delays. This method identified 807 instances of timing dependencies, with manual analysis confirming that 31 examined instances were true positives.

Several detection strategies have been developed to identify order-dependent tests, which are sensitive to the sequence in which they are executed. **Brittle Assertions** techniques detect brittle assertions that can lead to order-dependent tests. Tools like *OraclePolish* use taint analysis to identify inputs that may affect assertions. In a dataset of 13,609 test cases from various Java projects, this approach identified 164 brittle assertions. **Pollution Detection** identifies tests that leave side effects, which can cause order dependencies. Tools like *PolDet* model the internal memory state and compare it before and after test execution. In a dataset of 6,105 test methods from 26 open-source projects, 324 heap-polluting tests and 8 tests that polluted the file system were detected.

Dependency Graphs involve tools like *ElectricTest* and *PraDeT*, which build dependency graphs of tests to identify order dependencies. By analyzing read/write operations on shared resources, these tools validate dependencies through test execution. When evaluated on the same four developer-written test suites as *DTDetector*, these tools detected all the order-dependent tests identified by *DTDetector*, along with additional tests. The **iDFlakies Framework** detects flaky tests by executing test suites in modified orders (e.g., reversed or randomized) and comparing outcomes to identify order-dependent tests. Using a dataset of 183 open-source Java projects and an extended set of 500 GitHub projects, the framework identified 213 order-dependent flaky tests and 209 non-order-dependent flaky tests across 111 modules of 82 projects.

Finally, **Web Application-Specific Techniques** such as *TEDD* focus on web applications, using natural language processing to analyze test case names and identify potential dependencies based on input submissions. Evaluated on six open-source web application projects, *TEDD* identified between 32 and 34 flaky tests, depending on the configuration.

2.2 Machine Learning Approaches

Several studies have applied machine learning models to predict flaky tests based on both static and dynamic features of test cases. Models such as *FlakeFlagger* have shown promising results in predicting flaky tests without requiring repeated test executions.

Several studies have applied machine learning models to detect flaky tests, using different approaches and features. **King et al.** [16] utilized a Bayesian Network model incorporating both static and dynamic features, including complexity, implementation coupling, non-determinism, performance, and stability. The model was trained with historical data, such as average execution time, number of assertions, and failure rates. This approach achieved a prediction accuracy of 66% using historical instances of flaky tests.

Bertolino et al. [17] developed a technique called *FLAST*, which used a k-nearest neighbor (k-NN) classifier with static features derived from test case source code using the bag-of-words technique. This method reported a mean

hyperplane to separate classes in high-dimensional space. SVM is robust against noisy datasets and problems involving multi-feature interactions, making it a strong candidate for this study. Finally, the **Neural Network (Multilayer Perceptron)** was selected for its capacity to capture intricate patterns and dependencies in data. MLPs, which consist of an input layer, hidden layers, and an output layer, are effective for modeling complex relationships in large datasets [22].

3.3 Data Selection Process Phase

To ensure the validation and reliability of the proposed study in identifying and labeling flaky tests, it was essential to eliminate any ambiguity in the dataset regarding which tests were non-flaky. Several steps were taken to guarantee the accuracy and consistency of the dataset.

First, an **Initial Identification** phase was conducted, where ten runs were performed to identify flaky tests, with each test executed under the same conditions to maintain uniformity in detecting flakiness. Following this, **Developer Verification** was carried out. The developers of the six projects reviewed all potentially flaky tests identified in the initial phase to ensure that any observed non-deterministic behavior was due to the test being flaky and not caused by software bugs. Finally, the **Dataset Composition** phase involved finalizing the dataset after the developer verification. The final dataset included 793 verified flaky tests and 1707 confirmed stable tests.

These steps were thoroughly studied until the ground truth regarding flakiness was accurately established, ensuring that it could be considered for the proposed study.

3.4 Feature Extraction Phase

The proposed study considered both static and dynamic features, which were extracted to effectively characterize each test. This process helped create a rich database that was used for machine learning models to predict, analyze, and identify flaky tests.

Static features are characteristics of a test that remain fixed and independent of the test execution. The **Cyclomatic Complexity** measures the number of decision points in the code, such as if-else and loop statements. A higher cyclomatic complexity indicates more paths and greater complexity in the test. **Line of Code (LOC)** refers to the number of lines of code in a test. Larger tests may be flaky due to factors such as increased complexity or a broader scope. **Test Dependencies** highlight functions or classes that a test depends on. Tests with more dependencies are more susceptible to external changes, introducing non-deterministic behavior and increasing the likelihood of flakiness. **Sleep or Wait Calls** identify whether explicit timing control mechanisms, such as sleep or wait statements, are used. Tests with such timing controls are potentially flaky due to timing issues. Finally, **Assertions** refer to the type and number of assertions made by a test. The reliability of tests can be influenced by the

number and type of assertions, as a high number of assertions may make tests more sensitive to timing or environmental factors.

Dynamic features are characteristics that can fluctuate based on the test environment or execution conditions. The **Historical Pass Rate** is calculated by analyzing the results of previous test executions to determine the percentage of successful runs. A lower pass rate may indicate flaky behavior. **Execution Time Variability** refers to the standard deviation (SD) of execution times across multiple runs, which helps identify any instability in the test's performance. **Resource Usage** tracks patterns in CPU and memory consumption during test execution. Significant fluctuations or unusual resource consumption may signal underlying issues that contribute to flaky test results. Lastly, **Environment Variables** encompass factors like the operating system and hardware configurations, which can influence the test outcomes and may be a source of test flakiness.

By combining these static and dynamic features, the feature extraction process provides a comprehensive and detailed characterization of each test. These features are critical for training machine learning models to accurately identify and predict flaky tests, thereby enhancing the reliability and efficiency of software testing.

3.5 Model Training and Tuning Phase

The data set was split into training, validation, and test sets using a stratified sampling approach, ensuring a balanced distribution of flaky and nonflaky tests. The split was 80/10/10 for training, validation and testing, respectively. To avoid overfitting, we used a k-fold cross-validation (with k=5) during the training phase. Regularization techniques such as L1/L2 penalties were applied to reduce the complexity of the model and improve generalization. Early stopping was implemented to stop training when performance in the validation set plateaued.

For each model, the hyperparameters were fine-tuned using grid search: The **Decision Tree** model had its maximum depth fine-tuned from 5 to 15, while the minimum sample split was adjusted between 2 and 5. For the **Random Forest** model, 100 trees were used with a maximum depth of 15. Hyperparameters, including the number of trees and the maximum depth, were optimized through grid search. The **SVM** model's performance was improved by tuning hyperparameters such as the regularization parameter C, gamma, and kernels. Lastly, the **Neural Network** model employed a structure with 2 hidden layers, each containing 64 neurons. The batch size, the number of hidden layers, and the number of neurons per layer were optimized during the tuning process to enhance performance.

3.6 Performance Evaluation Phase

We used several performance metrics to evaluate the models.

Precision refers to the proportion of true positive predictions among all positive predictions. **Recall** is the proportion of true positive predictions among all

actual positives. **F1-Score** is the harmonic mean of precision and recall, offering a balanced measure of model performance. Finally, **ROC-AUC** represents the area under the receiver operating characteristic curve, evaluating the trade-off between the true positive rate and the false positive rate.

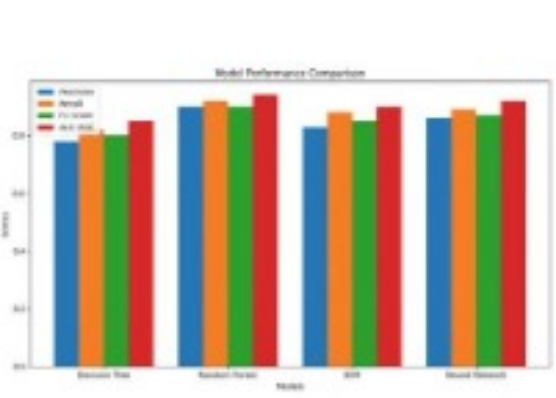
The combination of these metrics provides a comprehensive view of model performance, balancing the need for both accuracy (precision) and recall.

4 Results and Discussions

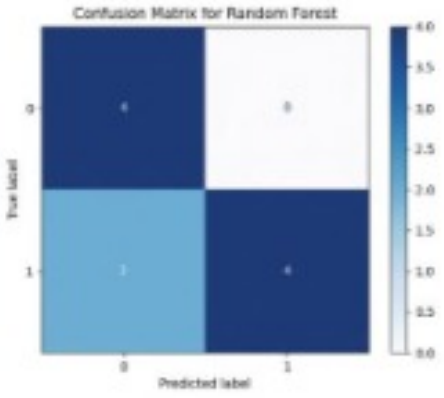
The implementation results of the machine learning models are summarized in Table 1. These metrics measure the efficiency of each model in various aspects of performance. The CI (95%) column represents the confidence intervals for each metric, calculated from cross-validation.

Table 1. Model Performance Comparison with Confidence Intervals

Model	Precision	Recall	F1-Score	AUC-ROC	CI (95%)
Decision Tree	0.78 ± 0.03	0.82 ± 0.02	0.80 ± 0.02	0.85 ± 0.02	[0.74, 0.82]
Random Forest	0.90 ± 0.02	0.92 ± 0.02	0.90 ± 0.01	0.94 ± 0.01	[0.88, 0.92]
SVM	0.83 ± 0.03	0.88 ± 0.02	0.85 ± 0.02	0.90 ± 0.02	[0.80, 0.85]
Neural Network	0.86 ± 0.02	0.89 ± 0.01	0.87 ± 0.01	0.92 ± 0.01	[0.83, 0.88]



(a) Model Performance Comparison (Precision, Recall, F1-Score, AUC-ROC)



(b) Confusion Matrix for Random Forest Model

Fig. 1. Comparison of Models and Confusion Matrix.

Among the models evaluated, the **Random Forest** model achieved the highest overall performance across all metrics. Specifically, it demonstrated an accuracy of 91%, a precision of 90%, a recall of 92%, and an F1-score of 94%. The high

performance of the Random Forest can be attributed to its ability to handle complex interactions between features and reduce overfitting through ensemble techniques.

In comparison, the **Neural Network** model showed slightly lower performance but still outperformed the other models except for Random Forest. The complexity and extensive tuning required for Neural Networks may have contributed to its slightly lower performance relative to Random Forest.

The **Decision Tree** and **Support Vector Machine (SVM)** models performed reasonably well but did not match the performance of the Random Forest and Neural Network models. Decision Trees tend to overfit, especially when dealing with high-dimensional data, while SVMs require careful tuning of hyperparameters, which can be challenging in practice.

In summary, the Random Forest model was the best performing model, followed by the Neural Network model, both of which showed similar task-specific outcomes. Both models demonstrated a high ability to detect flaky tests, although the Random Forest model slightly outperformed the Neural Network in overall performance. The SVM model showed great results and can be applied depending on specific needs and constraints. The Decision Tree, while useful for initial insights and interpretability, has lower accuracy and robustness compared to the other models. The results show that combining ensemble methods with advanced pattern recognition abilities helps address the issue of detecting flaky tests more effectively.

The results indicate that machine learning models, particularly Random Forests, are highly effective in detecting flaky tests within automated regression suites. The high accuracy, precision, recall, and F1-score achieved by the Random Forest model demonstrate its robustness and ability to generalize well to unseen data. The ensemble nature of Random Forest allows it to capture complex interactions between features and reduce overfitting, making it particularly suitable for this task. On the other hand, while Neural Networks showed strong performance, they required more computational resources and tuning compared to Random Forests. Decision Trees and SVMs, although simpler and faster to train, did not achieve the same level of performance, likely due to their susceptibility to overfitting and the need for careful hyperparameter tuning.

Despite promising results, our study has some limitations. First, the quality and completeness of the training data is crucial for model performance. In some cases, tests with low code coverage or insufficient historical data may not be accurately classified, leading to lower model performance. Additionally, the complexity of certain models, such as Neural Networks, requires significant computational resources and expertise for tuning, which may not be feasible for all development teams. Another limitation is the potential bias in the data set. If training data consists predominantly of tests from a specific type of environment or project, the models may not generalize well to other contexts. Ensuring a diverse and representative dataset is essential for building robust models.

5 Conclusions

Our research aimed to develop machine learning models capable of accurately identifying flaky tests within automated regression suites. Through extensive data collection, feature engineering, and model evaluation, we demonstrated that Random Forest models outperformed other algorithms in terms of accuracy, precision, recall, F1 score, and ROC-AUC. Neural networks also showed strong performance, but required more computational resources and tuning. Decision trees and SVMs, while simpler, did not achieve the same level of accuracy.

In case studies involving real-world projects, our models successfully identified previously undetected flaky tests, leading to improved reliability and efficiency of the regression testing process. These findings underscore the potential of machine learning techniques to address the significant challenge of test automation flakiness.

Our study evaluated four ML models, with Random Forest achieving the highest performance, boasting a precision of 0.90 and recall of 0.92, resulting in an F1 score of 0.90. This outperforms previous models such as King et al. Bayesian Network (66% accuracy), Bertolino et al.'s k-NN (F1 score 0.60), and Haben et al.'s time-sensitive approach (average F1 score 0.80). Our results are competitive with Pinto et al. Random Forest (F1 score 0.95) and exceed Alshammari et al.'s combined feature Random Forest (F1 score 0.86), demonstrating significant improvements in flaky test detection.

References

1. Pachouly, J., Ahirrao, S., Kotecha, K., Selvachandran, G., Abraham, A.: A systematic literature review on software defect prediction using artificial intelligence: datasets, data validation methods, approaches, and tools. Eng. Appl. Artif. Intell. **111**, 104773 (2022)
2. Parry, O., Kapfhammer, G.M., Hilton, M., McMinn, P.: A survey of flaky tests. ACM Trans. Softw. Eng. Methodol. **31**(1), 1–74 (2021)
3. Lam, W., Muşlu, K., Sajnani, H., Thummalapenta, S.: A study on the lifecycle of flaky tests. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, pp. 1471–1482 (2020)
4. Zolfaghari, B., Parizi, R.M., Srivastava, G., Hailemariam, Y.: Root causing, detecting, and fixing flaky tests: state of the art and future roadmap. Softw. Pract. Exp. **51**(5), 851–867 (2021)
5. Kowalczyk, E., Nair, K., Gao, Z., Silberstein, L., Long, T., Memon, A.: Modeling and ranking flaky tests at apple. In: 2020 IEEE/ACM 42nd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp. 110–119. IEEE (2020)
6. Gonze, X., et al.: The ABINIT project: impact, environment and recent developments. Comput. Phys. Commun. **248**, 107042 (2020)
7. Dutta, S., Shi, A., Choudhary, R., Zhang, Z., Jain, A., Misailovic, S.: Detecting flaky tests in probabilistic and machine learning applications. In: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 211–224 (2020)